S-AES 开发手册

一、 项目概览

本项目使用 Python 标准库实现 S-AES，并通过 Tkinter 提供图形界面。系统按“算法核心—界面编排—测试与报告”分层，支持单分组加解密、ASCII 字符串加解密、双重与三重加密、中间相遇攻击，以及 CBC 模式与密文篡改实验。

算法模块不依赖图形界面，可由测试程序或其他 Python 程序独立调用，便于验证算法正确性和后续扩展。

```
S-AES/
├── app.py            Tkinter 图形程序入口
├── saes.py           S-AES 算法、字节处理、多重加密、攻击与 CBC
├── test_saes.py       unittest 自动化测试
├── README.md          运行说明与算法约定
├── .gitignore         缓存及临时文件忽略规则
└── file/              用户指南、测试结果等项目文档
```

二、 运行与测试

项目兼容 Python 3.10+，只使用标准库，不需要安装第三方 Python 包。图形界面要求运行环境包含 Tkinter。

在项目根目录执行以下命令启动程序：

```
python app.py
```

执行自动化测试：

```
python -m unittest -v
```

如需避免生成字节码缓存，可执行：

```
python -B -m unittest -v
```

三、 算法模块设计

1. 数据表示

saes.py 使用无符号整数表示分组和密钥，使用 bytes 表示字节消息。界面将二进制文本解析为整数，输出时按固定宽度转换为二进制和十六进制，保留前

导零。

基础分组和密钥长度均为 16 bit。设分组由高到低的四个半字节为 n0、n1、n2、n3，状态矩阵排列为：

$$\begin{bmatrix} n0 & n2 \\ n1 & n3 \end{bmatrix}$$

S 盒定义为：

```
9 4 A B D 1 8 5 6 2 0 3 C E F 7
```

逆 S 盒根据 S 盒自动生成。有限域乘法使用模多项式 $x^4 + x + 1$，即 0x13；列混淆矩阵为 $\begin{bmatrix} 1 & 4 \\ 4 & 1 \end{bmatrix}$，逆矩阵为$\begin{bmatrix} 9 & 2 \\ 2 & 9 \end{bmatrix}$。

2. 核心接口

函数	输入	输出	说明
`parse_bits(text, bits=16)`	二进制文本、位宽	整数	解析并校验二进制输入
`gf_multiply(left, right)`	两个 4 bit 元素	4 bit 结果	实现有限域乘法
`substitute(state, box=SBOX)`	状态、替换表	16 bit 状态	半字节替换
`shift_rows(state)`	16 bit 状态	16 bit 状态	交换第二行两个半字节
`mix_columns(state,inverse=False)`	状态、逆变换标志	16 bit 状态	列混淆或逆列混淆
`expand_key(key)`	16 bit 密钥	(K0, K1, K2)	生成轮密钥
`encrypt_block(plaintext, key)`	16 bit 明文、密钥	16 bit 密文	单分组加密
`decrypt_block(ciphertext, key)`	16 bit 密文、密钥	16 bit 明文	单分组解密

密钥扩展使用轮常量 0x80 和 0x30，并通过 `lru_cache` 缓存扩展结果。示例：

```
2D55 → BCE9 → A34A
```

单分组加密流程为：

```
P → AK(K0) → NS → SR → MC → AK(K1) → NS → SR → AK(K2) → C
```

其中，AK 为轮密钥加，使用异或实现；NS 为半字节替换；SR 为行移位；MC 为列混淆。

解密按相反顺序执行对应逆变换。由于第二行移位只涉及两个半字节的交换，

行移位与逆行移位复用 `shift_rows()`。

3. 字符串与多重加密

函数	输入	输出	说明
`split_key(key, count)`	组合密钥、子密钥数量	子密钥元组	从高位到低位拆分密钥
`multiple_encrypt(plaintext, key, count=2)`	明文、组合密钥、重数	16 bit 密文	双重或三重加密
`multiple_decrypt(ciphertext, key,count=2)`	密文、组合密钥、重数	16 bit 明文	逆序解密
`pad(data)`	字节序列	填充后的字节序列	添加 PKCS#7 填充
`unpad(data)`	带填充字节序列	原始字节序列	校验并移除填充
`transform_bytes(data,key, decrypt=False)`	字节序列、密钥	字节序列	独立分组加解密

双重加密采用 32 bit 组合密钥，左侧为 K_1，右侧为 K_2：

$$C = E(K_2, E(K_1, P))$$

三重加密采用 48 bit 组合密钥：

$$C = E(K_3, E(K_2, E(K_1, P)))$$

解密按相反顺序使用各子密钥。多重单分组操作不执行填充。

字符串由界面严格按 ASCII 编码，再按大端序每 2 字节组成一个分组。PKCS#7 填充规则为：奇数字节长度补一个 01，偶数字节长度补两个 02，空字符串也补为 0202。

密文以 HEX 显示和输入，同时提供 Latin-1 字节映射的 JSON 转义表示，避免控制字符直接影响显示。

4. 中间相遇攻击

`meet_in_the_middle(pairs, progress=None, cancelled=None)` 接收明密文对，返回满足全部观测的 32 bit 候选密钥列表。

根据双重加密关系：

$$C = E(K_2, E(K_1, P))$$

可得到：

$$E(K_1, P) = D(K_2, C)$$

程序首先枚举全部 65536 个 K_1，建立中间值到密钥列表的映射；再枚举全部 65536 个 K_2，查找相同中间值，最后利用其余明密文对过滤候选。

同一中间值对应的全部密钥都会保留，避免碰撞覆盖导致候选遗漏。搜索成本约为两轮 2^{16} 次枚举，加上候选验证开销，存储规模为 $O(2^{16})$。

单组明密文通常不能唯一确定组合密钥。返回结果表示候选满足已有观测，不能直接保证其就是原始密钥。

5. CBC 模式

函数	输入	输出	说明
`new_iv()`	无	16 bit 整数	生成随机 IV
`cbc_encrypt(data, key, iv)`	明文字节、密钥、IV	密文字节	填充后执行 CBC 加密
`cbc_decrypt(data,key,iv, remove_padding=True)`	密文字节、密钥、IV	明文字节	可选择保留填充

设 $C_0 = IV$，CBC 运算为：

$$C_i = E(K, (P_i \; XOR \; C_{i-1}))$$

$$P_i = D(K, C_i) \; XOR \; C_{i-1}$$

IV 使用 `secrets.randbits(16)` 生成，不自动拼入密文，需要单独共享。界面加密时使用当前 IV 输入框的值，每条新消息应重新生成 IV。

正常解密校验并移除填充；篡改对比保留填充，逐组显示原始解密字节。修改密文分组 C_i 会扰乱 P_i，使 P_{i+1} 对应位翻转，之后的分组恢复正常。修改 IV 只影响首个明文分组。

四、输入校验与异常

`check_word(value, bits=16)` 检查整数类型和位宽范围，并拒绝将布尔值作为合法整数。`parse_bits()` 去除空白后，检查输入长度以及是否只包含 0、1。

算法层还检查多重加密重数、密文长度、填充内容和攻击输入。非法参数使用 `ValueError` 报告，取消攻击使用 `InterruptedError` 报告。

界面通过 ASCII 编码检查明文字符范围，通过 bytes.fromhex() 解析密文。中间相遇页面要求每个非空行包含两项完整的 16 bit 二进制数据。

普通按钮通过 safe() 统一捕获 ValueError、UnicodeError 和 OSError，以对话框反馈错误。后台攻击的异常通过队列交回主线程，在状态区域显示。算法模块不依赖 Tkinter。

五、GUI 架构

Application 继承自 tk.Tk，使用 ttk.Notebook 组织四个页面：

```
Application
├── configure_styles()    统一主题、字体和控件状态
├── tab()                 创建可滚动页面
├── build_blocks()        基本与多重加密
├── build_text()          ASCII 字符串加解密
├── build_attack()        中间相遇攻击
├── build_cbc()           CBC 与篡改实验
└── poll_events()         接收后台任务消息
```

颜色和字体集中配置在 configure_styles()。界面采用浅色内容区和蓝色主操作按钮，结果区只读，并支持复制。页面可垂直滚动，多行文本控件保留自身滚动行为。

页面事件函数负责读取输入、调用算法和更新控件，密码运算保留在 saes.py。

中间相遇攻击使用后台线程，通过 queue.Queue 传递进度、结果和异常。主线程每 100 毫秒调用 poll_events() 更新界面，工作线程不直接操作 Tkinter 控件。

算法最多每 1024 次枚举检查一次取消标志。攻击完成后，界面最多显示前 200 个候选，完整结果可通过“导出全部候选”保存。

六、增加功能的建议

6.1 过程追踪

可新增 encrypt_block_with_trace()，返回轮密钥及各次变换后的状态，便于与课件逐步核对。

应保持现有 encrypt_block() 的返回值不变，通过新增接口提供追踪数据，

避免影响已有调用和测试。

6.2 文件和网络扩展

文件加密可按二进制读取数据，复用字节处理或 CBC 接口。网络扩展应明确消息格式，包括模式、IV、密文长度和编码方式。

当前 CBC 接口对整条输入进行填充。若增加流式文件处理，应单独维护链值，并只在消息末尾填充，不能将多个独立加密片段直接拼接。

文件读写、Socket 通信和 Tkinter 事件处理应分模块实现，耗时操作通过后台任务执行。

6.3 搜索规模扩展

当前攻击已具备后台执行、进度反馈和取消机制。若扩大搜索空间，可进一步考虑多进程、分段任务和磁盘存储。

扩展时应保留中间值的全部碰撞候选，并继续使用额外明密文对验证结果。

七、测试策略

`test_saes.py` 使用标准库 `unittest`，不依赖图形界面，覆盖以下内容：

测试类别	验证内容
参考示例	密钥扩展、替换、移位、列混淆与有限域乘法
已知答案	D728 使用 4AF5 加密得到 24EC
全明文空间	固定密钥下全部 65536 个明文往返，且密文无重复
多重与随机密钥	单重、双重、三重往返及子密钥顺序
字节消息	长度 0 至 33 的消息在独立分组和 CBC 下往返
CBC 篡改	密文位翻转和 IV 修改的影响范围
中间相遇	完整搜索包含原密钥，且全部候选满足观测
错误处理	非法输入、越界、无效填充、空观测及取消

新增算法功能时，应补充正常输入、边界输入和非法输入测试。修改 S 盒、状态排列、密钥扩展或填充规则后，应重新运行全部测试。

GUI 还需检查按钮操作、页面滚动、结果复制、进度更新、取消和导出。自

动化算法测试不能代替逐按钮界面验收。

跨组测试应使用独立实现，先统一算法、字节序、编码和填充约定，再核对相同输入的输出。

八、代码规范

- 函数和变量使用 `snake_case`，类使用 `PascalCase`。
- 算法常量使用全大写，例如 `SBOX`、`INV_SBOX`。
- 算法层使用整数和字节序列，二进制与 HEX 输出显式指定宽度。
- 字节序统一使用大端序，多重密钥从左到右排列为K_1、K_2、K_3。
- UI 事件负责交互，密码学运算放在 `saes.py`。
- 工作线程通过队列传递数据，Tkinter 控件由主线程更新。
- 注释说明算法意图和特殊约定，避免重复描述显而易见的代码。
- 修改接口或数据约定时，同步更新测试和 `README`。